

TP1. Calibration de volatilité locale

Question 1) : Tracer le smile à partir de la base de données de couples (K, prix d'option) (LIFFE).

Tout d'abord nous allons définir chacune des fonctions nécessaires pour tracer le smile.

```
def N(x):
    return 0.5 * (1 + erf(x / sqrt(2)))

def d1(S0, K, r, sigma, T, t=0.0):
    return (log(S0/K) + (r + 0.5*sigma**2)*(T-t)) / (sigma*sqrt(T-t))

def d2(S0, K, r, sigma, T, t=0.0):
    return d1(S0, K, r, sigma, T, t) - sigma*sqrt(T-t)

def Vega_BS(S0, K, r, sigma, T, t=0.0):
    d1_ = d1(S0, K, r, sigma, T, t)
    return S0 * sqrt(T - t) * (1/sqrt(2*pi)) * exp(-0.5 * d1_**2)

##### Fonctions pour Call #####

def V_BS_Call(S0, K, r, sigma, T, t=0.0):
    return S0 * N(d1(S0, K, r, sigma, T, t)) - K * exp(-r*(T-t)) * N(d2(S0, K, r, sigma, T, t))

def F_Call(M, S0, K, r, sigma, T, t=0.0):
    return V_BS_Call(S0, K, r, sigma, T, t) - M

def newton_Call(M, S0, K, r, T, t=0.0, epsilon=1e-6, max_iter=100):
    # --- Test d'arbitrage pour un CALL ---
    borne_inf = max(S0 - K * exp(-r*(T - t)), 0.0)
    borne_sup = S0
    if not (borne_inf <= M <= borne_sup):
        return float("nan")

    sigma = sqrt(2 * abs(log(S0/K) + r*(T-t)) / (T - t))

    while abs(F_Call(M, S0, K, r, sigma, T, t)) > epsilon:
        sigma = sigma - F_Call(M, S0, K, r, sigma, T, t) / Vega_BS(S0, K, r, sigma, T, t)

    return sigma
```

Ici, V_BS représente le prix de l'option théorique de Black-Scholes en $t=0$, $Vega_BS$ représente la dérivée partielle de V_BS par rapport à la volatilité, F représente la différence entre la valeur théorique de Black-Scholes V_BS et la valeur de marché M . Enfin, $newton$ calcule la volatilité implicite. Cette dernière vérifie bien, avant de commencer le calcul, que le prix tombe bien dans l'intervalle défini par les contraintes d'arbitrage suivantes :

$$\max(S_0 - K * e^{-rt}, 0) < V^{marché} < S_0$$

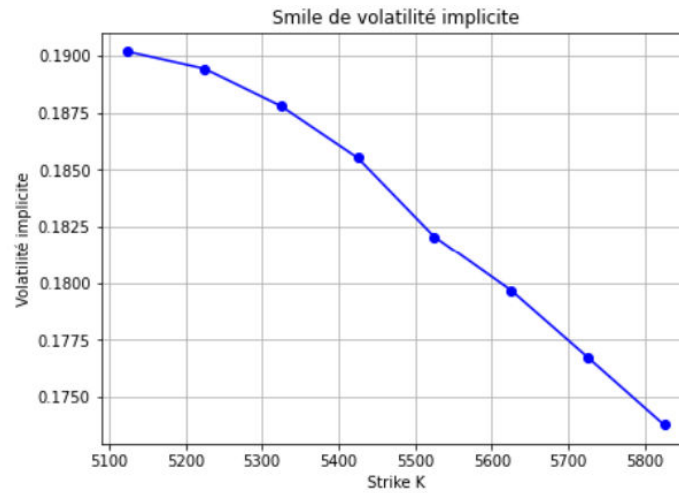
De plus, cette fonction prend bien comme point de départ :

$$\sigma_0 = \sqrt{2 * \left| \frac{\ln\left(\frac{S_0}{K}\right) + r * T}{T} \right|}$$

Ainsi, pour chaque strike K_i , on calcule la volatilité implicite, et on obtient :

	Strike	Prix marché	Vol implicite
0	5125	475.0	0.190177
1	5225	405.0	0.189431
2	5325	340.0	0.187792
3	5425	280.5	0.185525
4	5525	226.0	0.182066
5	5625	179.5	0.179687
6	5725	139.0	0.176755
7	5825	105.0	0.173786

Ainsi, on obtient le smile suivant :



Question 2) : Tracer les smiles pour Call et pour Put (sp-index avec des dividendes) séparément en 3 dimensions, puis les superposer à 3 dim.

Dans cette partie, nous allons prendre en compte le dividende fin de refléter au mieux la réalité. En effet, le calcul de la volatilité implicite des options est modifié par le paiement des dividendes par l'actif sous-jacent. Ainsi, nous remplaçons dans toutes les formules utilisées précédemment S_0 par $S_0 * e^{-q(T-t)}$ comme ceci :

```
sigma_imp_T = np.array([
    newton_Call(V, S0 * exp(-q * k), K_j, r_j, k)
    for K_j, r_j, V in zip(K_T, r_T, V_marche_T)
])
```

k étant ici la maturité.

De plus, pour les options Put, les fonctions utilisées précédemment changent, et on a donc finalement :

```
def V_BS_Put(S0, K, r, sigma, T, t=0.0):
    return K * exp(-r*(T - t)) * N(-d2(S0, K, r, sigma, T, t)) - S0 * N(-d1(S0, K, r, sigma, T, t))

def F_Put(M, S0, K, r, sigma, T, t=0.0):
    return V_BS_Put(S0, K, r, sigma, T, t) - M

def newton_Put(M, S0, K, r, T, t=0.0, epsilon=1e-6, max_iter=100):
    # --- test d'arbitrage pour un PUT ---
    borne_inf = max(K * exp(-r*(T - t)) - S0, 0.0)
    borne_sup = K * exp(-r*(T - t))
    if not (borne_inf <= M <= borne_sup):
        return float("nan")

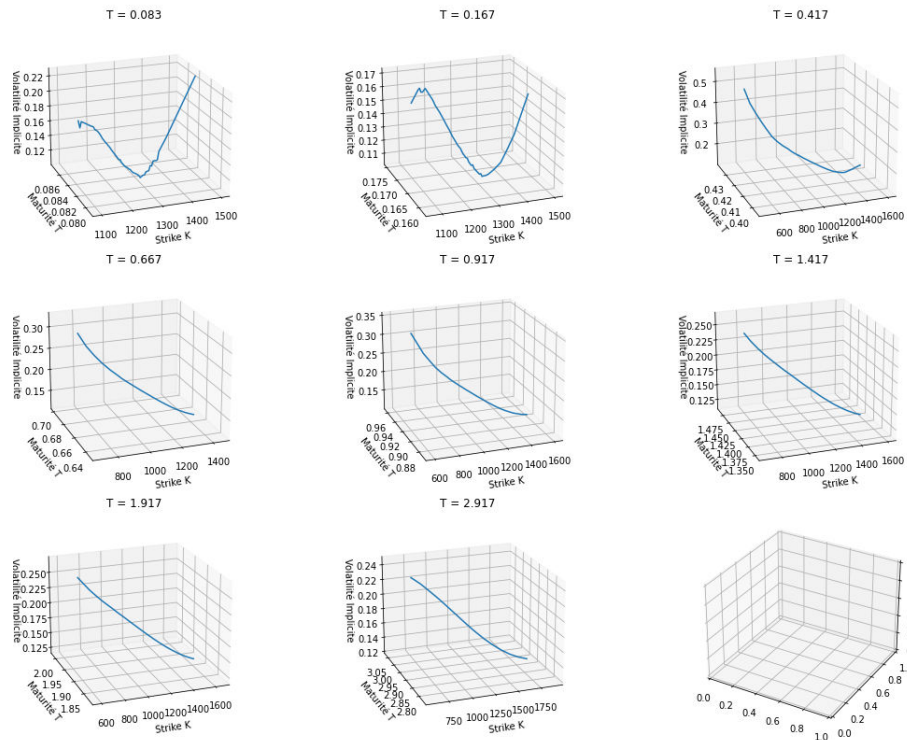
    sigma = sqrt(2 * abs(log(S0/K) + r*(T - t)) / (T - t))

    while abs(F_Put(M, S0, K, r, sigma, T, t)) > epsilon:
        sigma = sigma - F_Put(M, S0, K, r, sigma, T, t) / Vega_BS(S0, K, r, sigma, T, t)

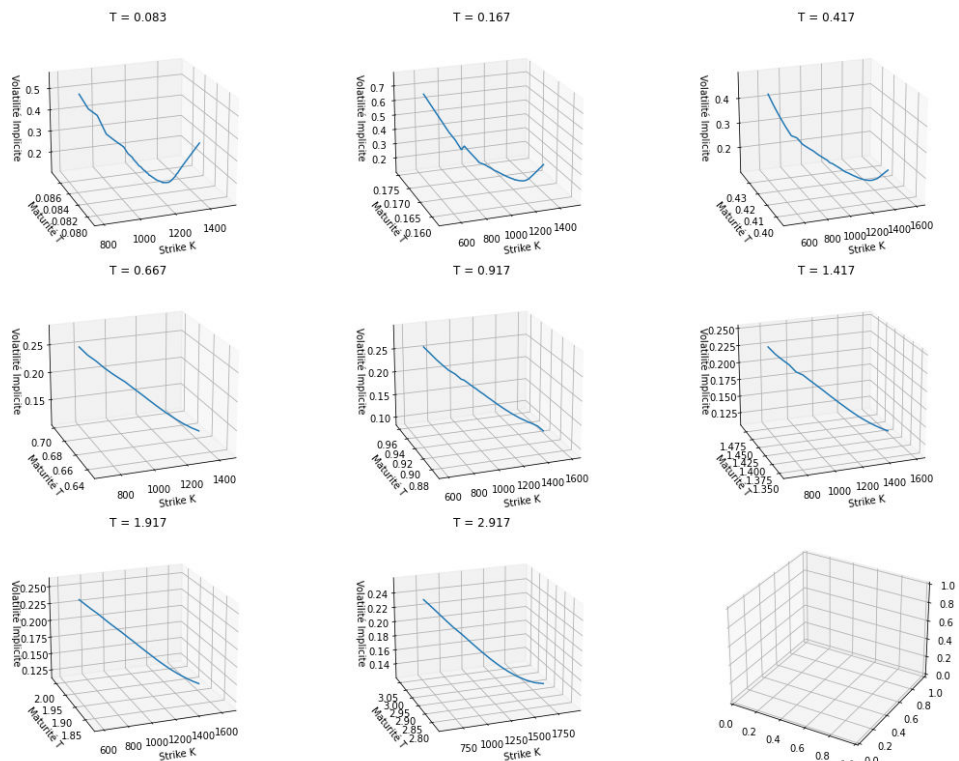
    return sigma
```

On obtient donc les smiles pour Call et Put suivant en 3D, affichés séparément :

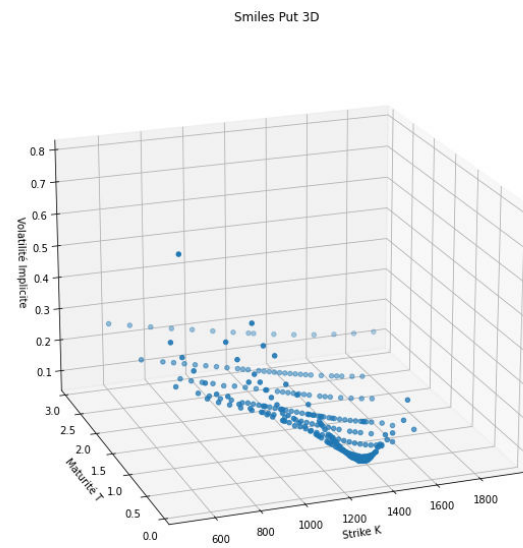
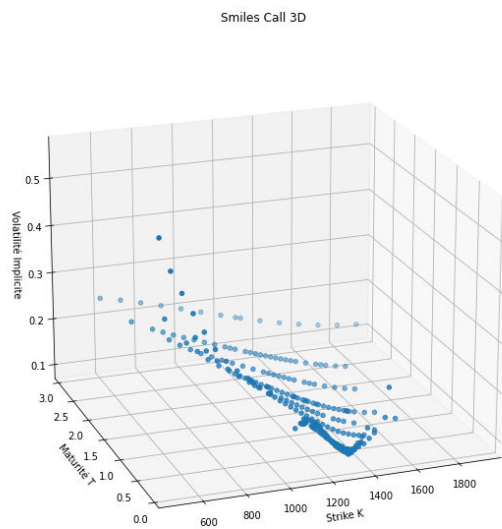
Smiles Call 3D séparés



Smiles Put 3D séparés

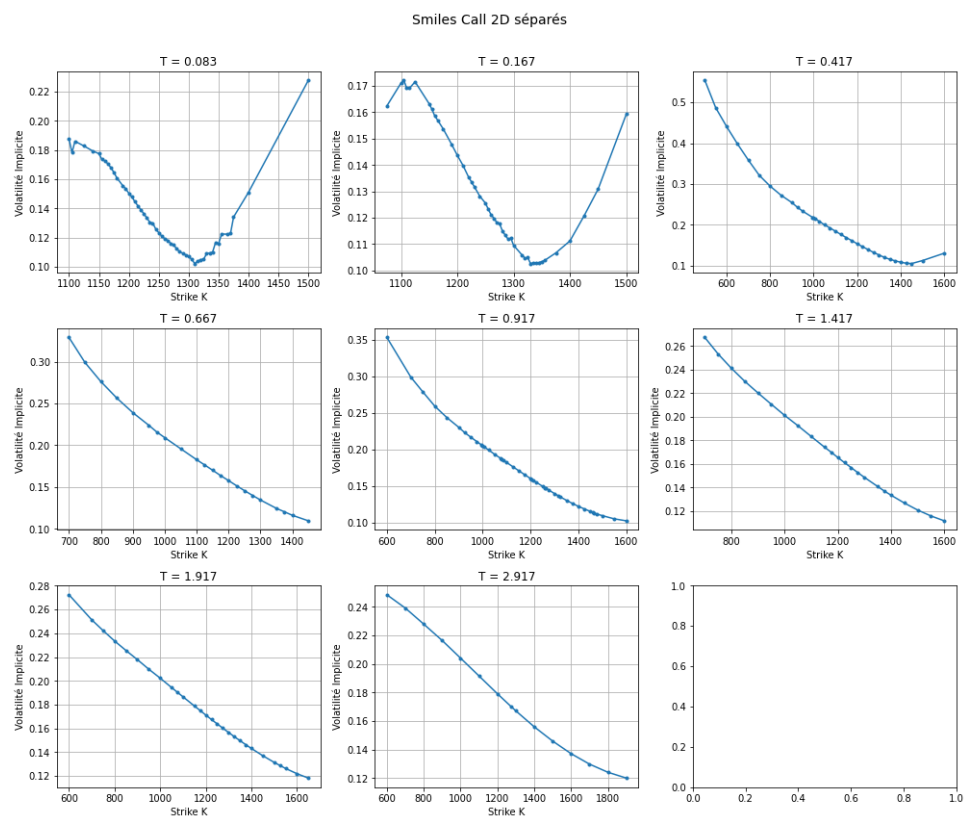


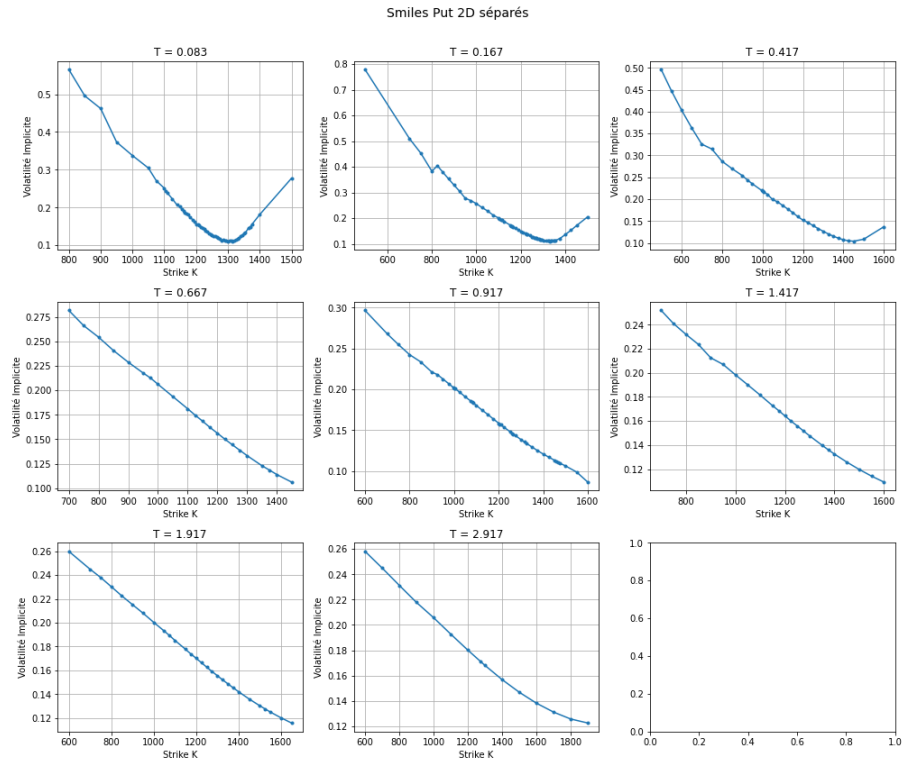
Les smiles Call et Put en 3D superposés sont les suivants :



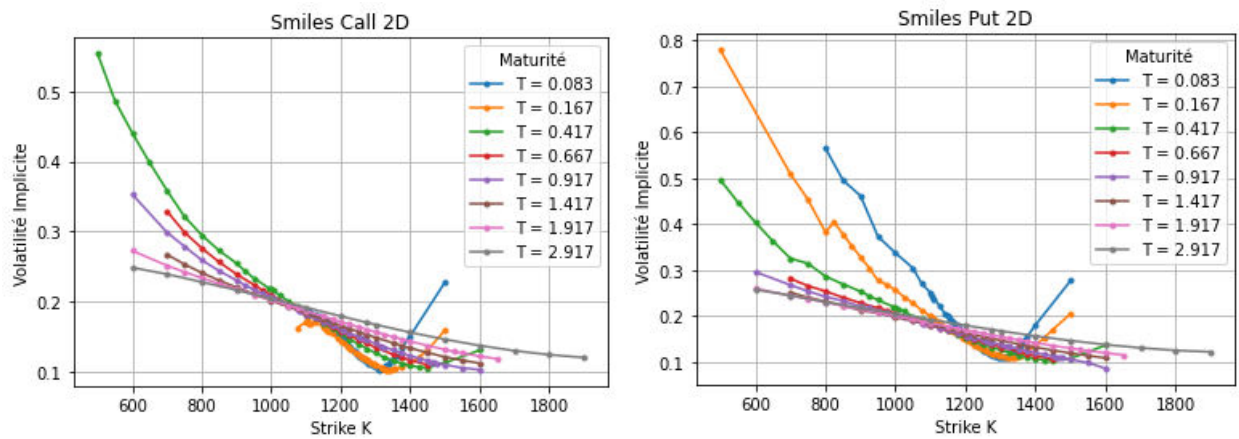
Question 3) : Tracer les smiles continus pour Call et pour Put de base (sp-index avec des dividendes) de données à 2 dim.

On obtient donc les smiles pour Call et Put suivant en 2D, affichés séparément :





Les smiles Call et Put en 2D superposés sont les suivants :



Question 4) : Calculer S_0 et le taux de dividende q . Tracer une droite de régression linéaire. Calculer S_0 et q à partir des formules théoriques.

Nous devons donc calculer S_0 et le taux de dividende q . Pour cela, nous allons chercher β_1 et β_2 en minimisant la somme des carrés des résidus comme ceci :

$$\min_{\beta^1, \beta^2} \Phi(\beta^1, \beta^2), \quad \Phi(\beta^1, \beta^2) = \sum (r_j)^2, \quad r_j = \ln \hat{S}_j - \beta^1 T_j - \beta^2$$

Pour cela, calculons tout d'abord les S_j _chapeau. Pour cela, nous allons appliquer la formule suivante :

$$\hat{S}_j = \frac{1}{N_j} \sum_{i=1}^{N_j} (Call(T_j, K_i) - Put(T_j, K_i) + e^{-r_j T_j} K_i)$$

Ce qui donne en code :

```
S_chapeau = []
T_j = []

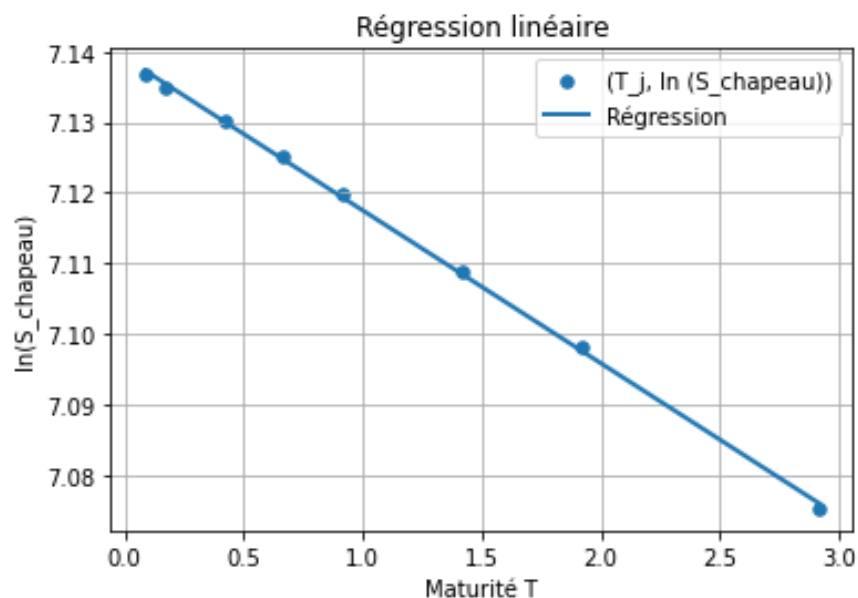
for tj in maturites_uniques:
    idx = np.where(T == tj)[0]
    Kj = K[idx]
    rj = rcol[idx]
    Cj = Call_mid[idx]
    Pj = Put_mid[idx]
    # moyenne sur tous les strikes disponibles à maturité tj
    S_chapeau_j = np.mean(Cj - Pj + np.exp(-rj * tj) * Kj)
    S_chapeau.append(S_chapeau_j)
    T_j.append(tj)

S_chapeau = np.array(S_chapeau, dtype=float)
T_j = np.array(T_j, dtype=float)
```

On trouve ensuite β_1 et β_2 par la fonction `np.polyfit` qui va calculer les coefficients du polynome de degré 1 qui approxime le mieux la relation entre deux vecteurs. On trouve donc :

```
Estimation (regression lineaire)
S0_reg = 1260.366679
q_reg = 0.021665
```

Ces résultats sont bien cohérents avec les valeurs censées être obtenues. On trace ensuite la droite de régression linéaire et on obtient :



Enfin, on calcule nos deux coefficients β_1 et β_2 avec les formules théoriques. On pose donc $x_j = T_j$ et $y_j = \ln(S_j_chapeau)$. Ensuite, on fait les caluls suivants :

Berkson
LECOEUR
ING3 MAF

On veut minimiser : $\phi(\beta_1, \beta_2) = \sum_{j=1}^n x_j^2$, $x_j = y_j - (\beta_1 x_j + \beta_2)$

avec $x_j = T_j$ et $y_j = \ln(\hat{S}_j)$

On le minimum d'une fonction $f(x)$ est atteint là où $f'(x) = 0$.

Ainsi, on regarde là où $\frac{\partial \phi}{\partial \beta_1} = 0$ et $\frac{\partial \phi}{\partial \beta_2} = 0$:

$$\begin{cases} \frac{\partial \phi}{\partial \beta_1} = -2 \sum_{j=1}^n x_j (y_j - \beta_1 x_j - \beta_2) = 0 \\ \frac{\partial \phi}{\partial \beta_2} = -2 \sum_{j=1}^n (y_j - \beta_1 x_j - \beta_2) = 0 \end{cases}$$

On pose : $S_x = \sum x_j$, $S_y = \sum y_j$, $S_{xx} = \sum x_j^2$, $S_{xy} = \sum x_j y_j$

Donc :
$$\begin{cases} \beta_1 S_{xx} + \beta_2 S_x = S_{xy} & (1) \\ \beta_1 S_x + \beta_2 n = S_y & (2) \end{cases}$$

On a donc d'après (2) : $\beta_2 = \frac{S_y - \beta_1 S_x}{n}$

et donc d'après (1) : $S_{xy} - \beta_1 S_{xx} - S_x \frac{S_y - \beta_1 S_x}{n} = 0$

$$\Leftrightarrow n S_{xy} - n \beta_1 S_{xx} - S_x S_y + \beta_1 S_x^2 = 0$$

$$\Leftrightarrow \beta_1 (n S_{xx} - S_x^2) = n S_{xy} - S_x S_y$$

On note le dénominateur $den = n S_{xx} - S_x^2$

Donc :
$$\begin{cases} \beta_1 = \frac{n S_{xy} - S_x S_y}{n S_{xx} - S_x^2} \\ \beta_2 = \frac{S_y - \beta_1 S_x}{n} \end{cases}$$

Et on retombe bien sur le même résultat donné dans le cours :

- $$\beta_1 = \frac{\sum_{j=1}^8 (T_j \cdot \ln(\hat{S}_j)) - (\sum_{j=1}^8 \ln(\hat{S}_j)) \cdot \sum_{j=1}^8 T_j}{(\sum_{j=1}^8 T_j^2) - (\sum_{j=1}^8 T_j)^2}$$
- $$\beta_2 = \frac{(\sum_{j=1}^8 T_j \cdot \ln(\hat{S}_j)) \cdot \sum_{j=1}^8 T_j - (\sum_{j=1}^8 \ln(\hat{S}_j)) \cdot \sum_{j=1}^8 T_j^2}{(\sum_{j=1}^8 T_j)^2 - \sum_{j=1}^8 T_j^2}$$

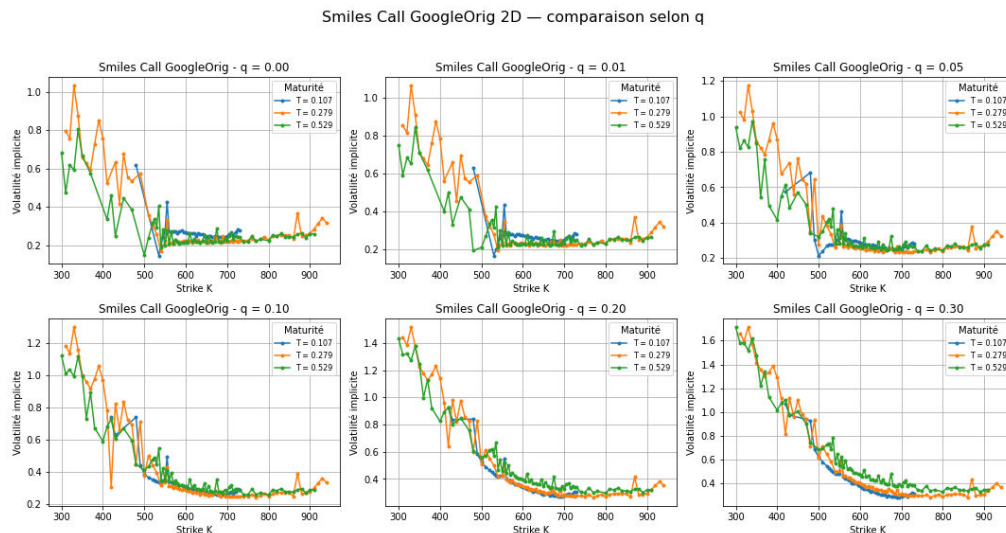
On obtient ainsi les résultats suivants :

Estimation par formules théoriques
beta1 = -0.02166467 donc q = 0.02166467
beta2 = 7.13915797 donc S0 = 1260.36667871

Ces résultats sont cohérents avec les valeurs censées être obtenues.

Question 5) : Tracer les smiles de la base **GoogleOrig.xlsx**, essayer le choisir le taux de dividende q pour obtenir des jolies smiles.

Ici, on nous impose les valeurs suivantes, $r=0.06$ et $S_0 = 591.66$. A partir de ces valeurs, le but est de « calibrer » notre taux de dividende q afin d'obtenir un smile « joli ». Ainsi, on créé la liste de taux de dividende q suivante $q_list = [0.00, 0.01, 0.05, 0.10, 0.20, 0.30]$ et on va tracer les smiles pour chaque q . On obtient donc ces smiles :



Ainsi, les smiles les plus jolis sont obtenus lorsque q est grand, où $q = 0.30$.

Question 6) : * Tracer les smiles de la base `spx_quotedata.csv` pour Call. Tracer séparément 13 premières smiles pour $q=0$. Expliquer clairement comment vous avez calculé les maturités à partir du traitement de la base de données. Calculer le taux de dividende q via la régression linéaire.

Il nous faut tout d'abord tracer les 13 premiers smiles pour $q=0$. Pour ceci, nous devons calculer les maturités pour chacun des calls. Ainsi, nous allons définir la date d'aujourd'hui comme étant la 1^{ère} date du fichier, c'est-à-dire le 13-09-2022. Ensuite, nous allons utiliser la formule suivante :

$$\frac{\text{Date d'expiration} - \text{date d'aujourd'hui}}{365.25}$$

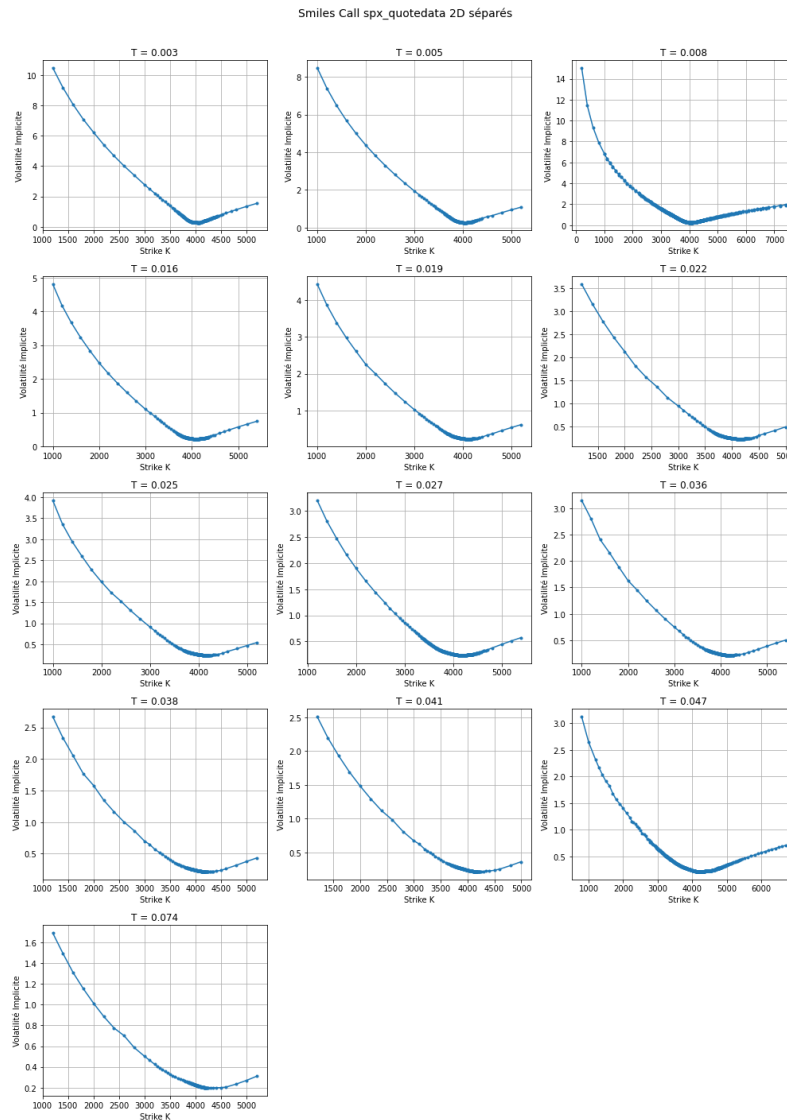
En définissant nos valeurs comme ceci :

```
S0 = 3932.69
q = 0.0
r = 0.0255 # taux constant

# --- passe en NumPy pour un indexage cohérent avec np.where ---
K = data.iloc[:, 11].to_numpy(float)
Cb = data.iloc[:, 4].to_numpy(float)
Ca = data.iloc[:, 5].to_numpy(float)
Pb = data.iloc[:, 15].to_numpy(float)
Pa = data.iloc[:, 16].to_numpy(float)

Call_mid = (Cb + Ca) / 2.0
Put_mid = (Pb + Pa) / 2.0 # (si utile ensuite)
```


Nous avons les 13 premiers smiles suivants :



Enfin, via la régression linéaire, on obtient que :

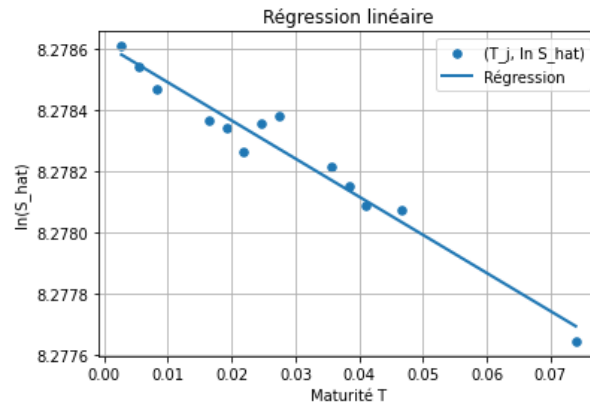
```
Estimation (régression linéaire via parité Call-Put)
S0_reg = 3938.744166
q_reg = 0.012487

Estimation par formules théoriques
beta1 = -0.01248727 donc q = 0.01248727
beta2 = 8.27861721 donc S0 = 3938.74416573
```

Donc en effet notre $S_0 \approx 3932.69$ et $q \sim 0$. L'écart entre le S_0 estimé et celui observé est extrêmement faible, ce qui confirme la cohérence de la régression et la validité du modèle. En effet, $\frac{3938.74 - 3932.69}{3932.69} \approx 0.15\%$.

Question 7) : Tracer la droite de la régression linéaire et l'analyser.

On obtient donc la droite de régression linéaire suivante



La régression linéaire entre $\ln(S_{\text{chapeau}})$ et T permet d'estimer un taux de dividende $q > 0$ cohérent avec les données du marché.

Le coefficient directeur négatif indique une décroissance légère de S_{chapeau} avec la maturité, conforme à la théorie. Plus la maturité est longue, plus la valeur actualisée des flux futurs diminue légèrement.

Le signe de la pente confirme donc la présence d'un flux de dividendes intégré dans les prix d'options.

L'écart entre le S_0 estimé et celui observé est extrêmement faible, ce qui confirme la cohérence de la régression et la validité du modèle.

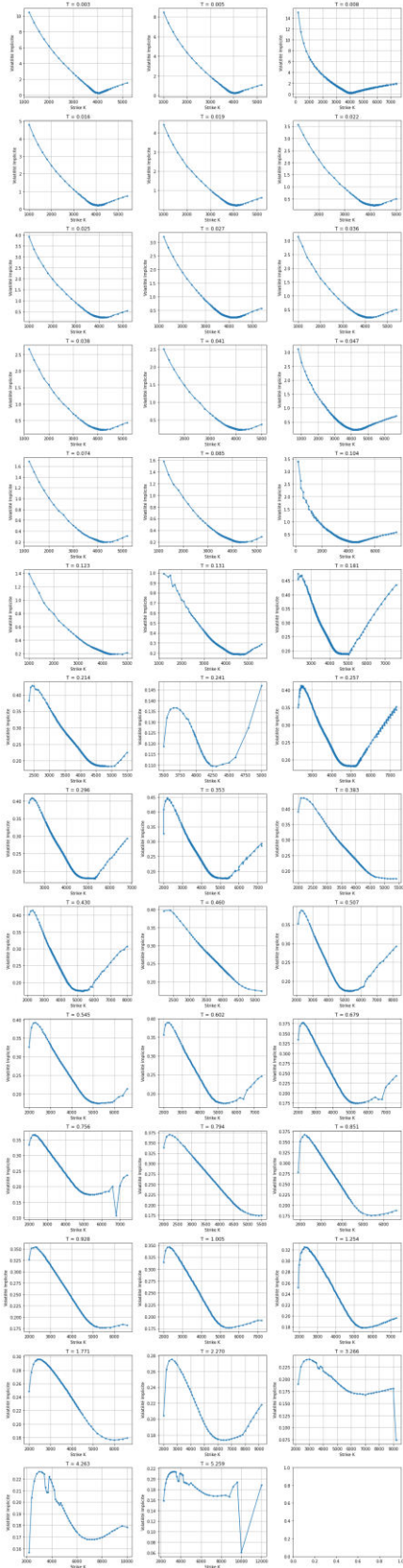
Dans l'ensemble, la qualité de l'ajustement est excellente et confirme la validité de la parité call-put et du modèle de dividende continu sur les 13 premières maturités analysées. Visuellement, les points sont très bien alignés le long de la droite, ce qui montre que la relation linéaire est très bien vérifiée.

Les écarts mineurs autour de la droite s'expliquent par le bruit de marché.

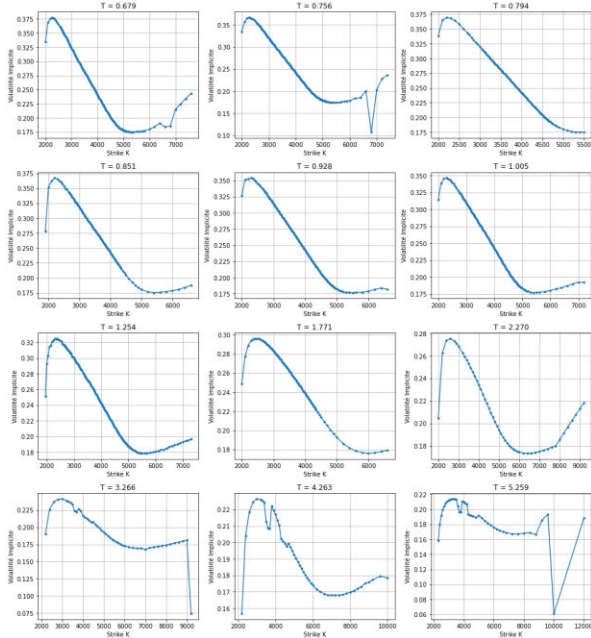
Question 8) : Tracer tout les smiles pour Call. Tracer les 12 derniers smiles pour Call.

A gauche on retrouve tout nos smiles Pour les Calls, et à droite on retrouve nos 12 derniers Smiles pour Call.

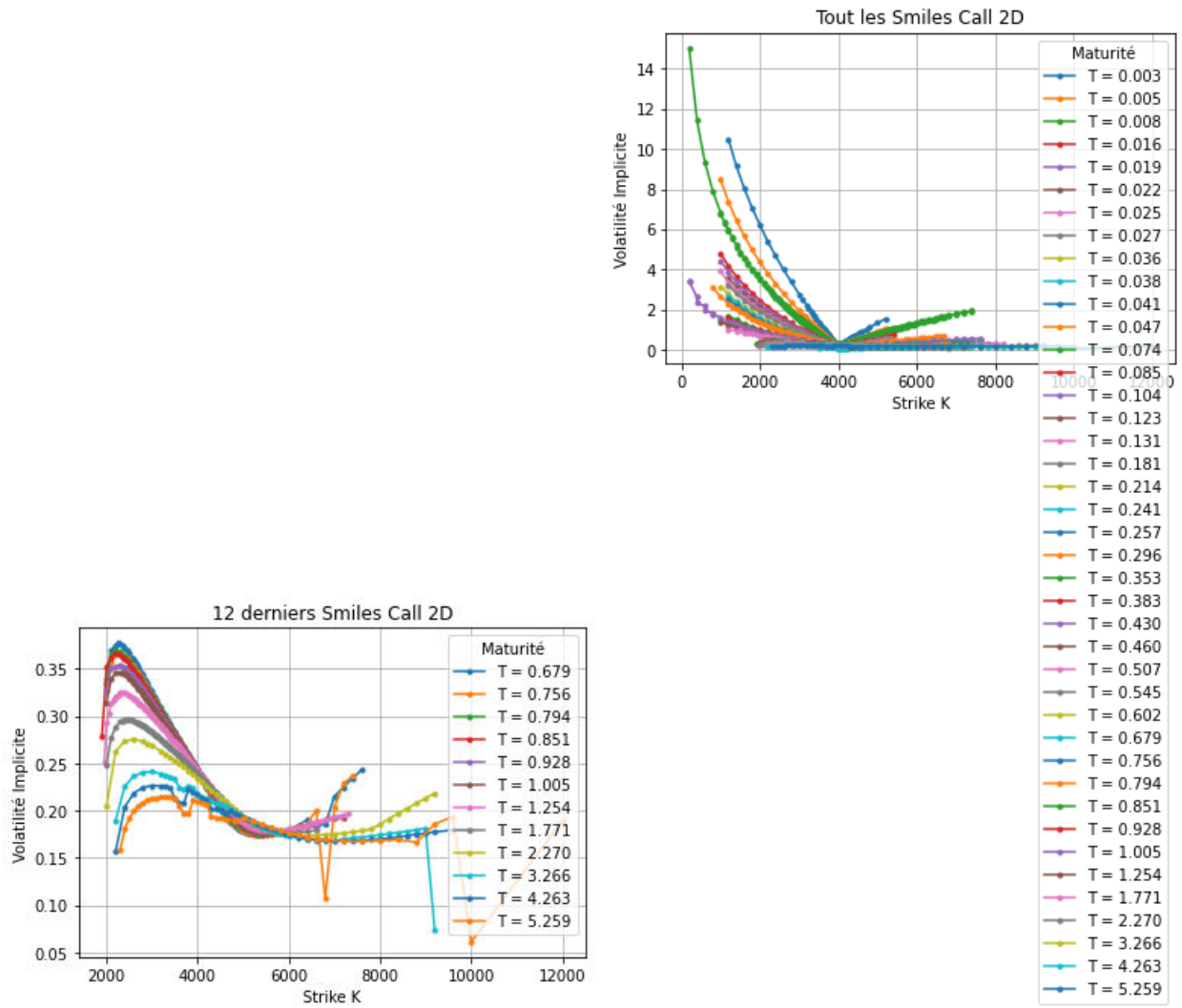
Smiles Call spx_quotedata 2D séparés



Smiles Call spx_quotedata 2D séparés



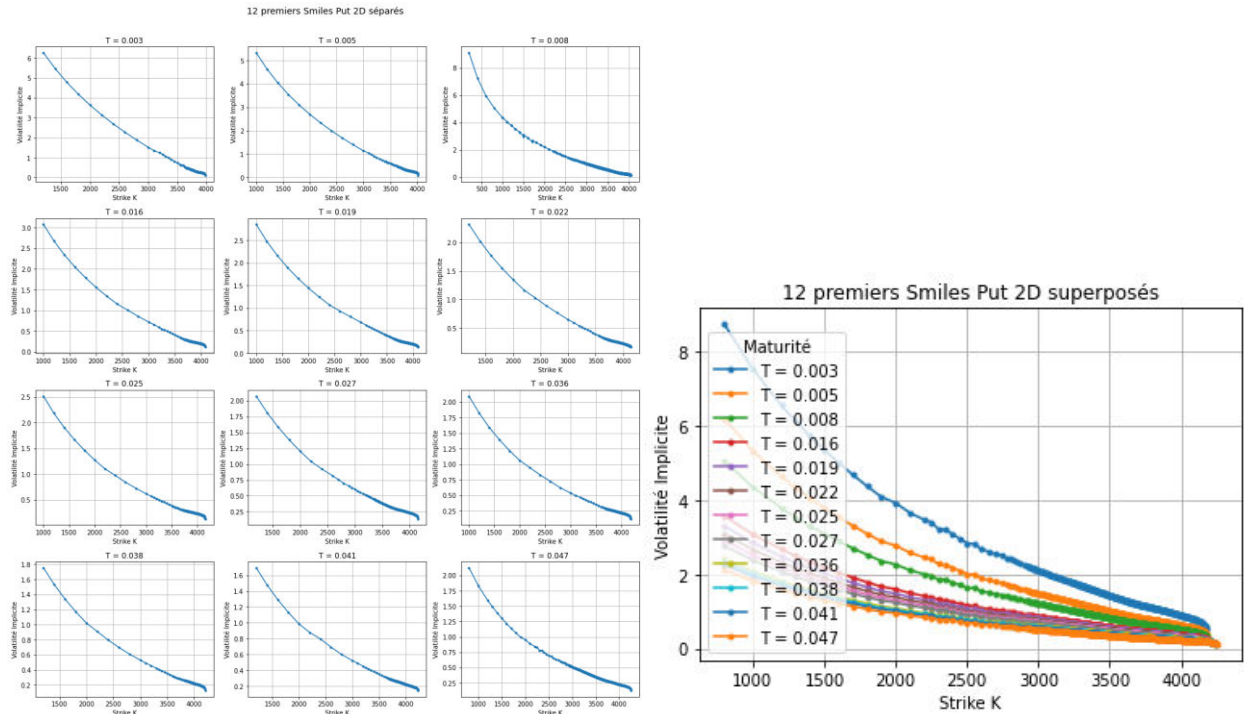
Ici, on les superposent tous, on a donc à gauche nos 12 derniers smiles et à gauche tout nos smiles



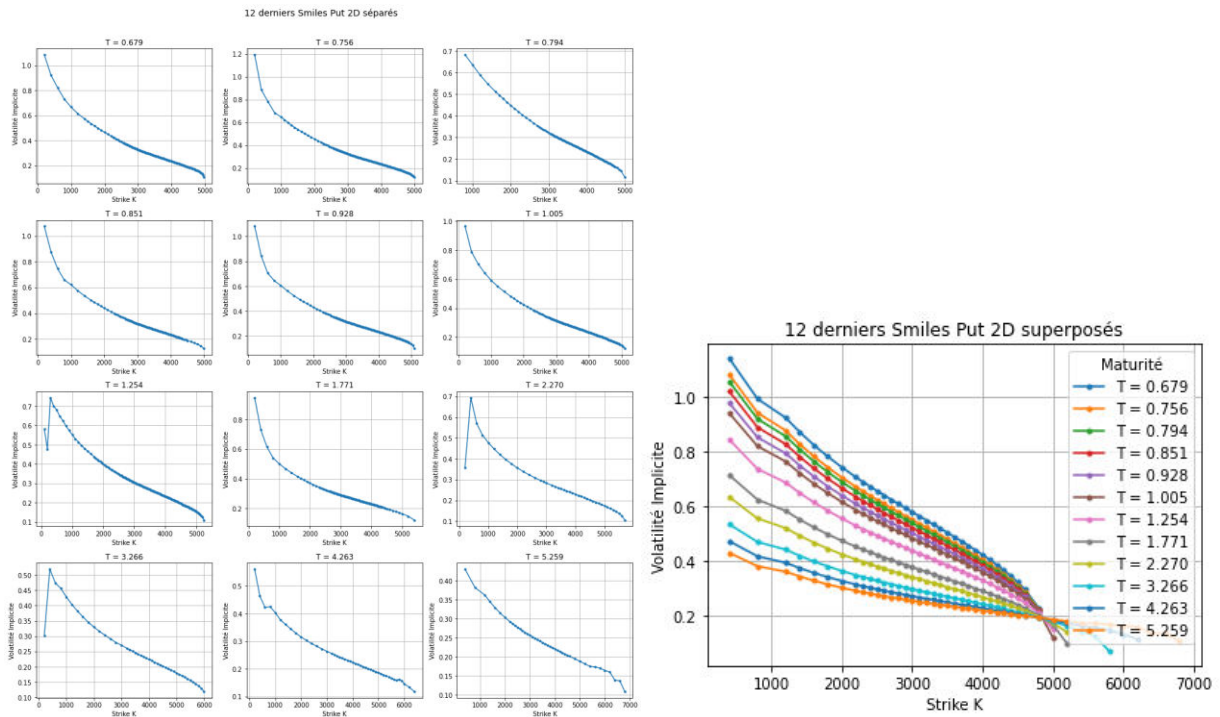
Question 9) : Tracer 12 premières et 12 dernières smiles pour Put

On applique donc la même méthode, mais pour les Puts. On utilise maintenant la fonction `newton_Put`.

On obtient donc les 12 premiers smiles pour Put suivants :



Et on obtient donc les 12 derniers smiles pour Put suivants :



Question 10) : Tracer les smiles pour new-Data.csv, $r=5\%$. Tracer la droite de la régression linéaire et l'analyser.

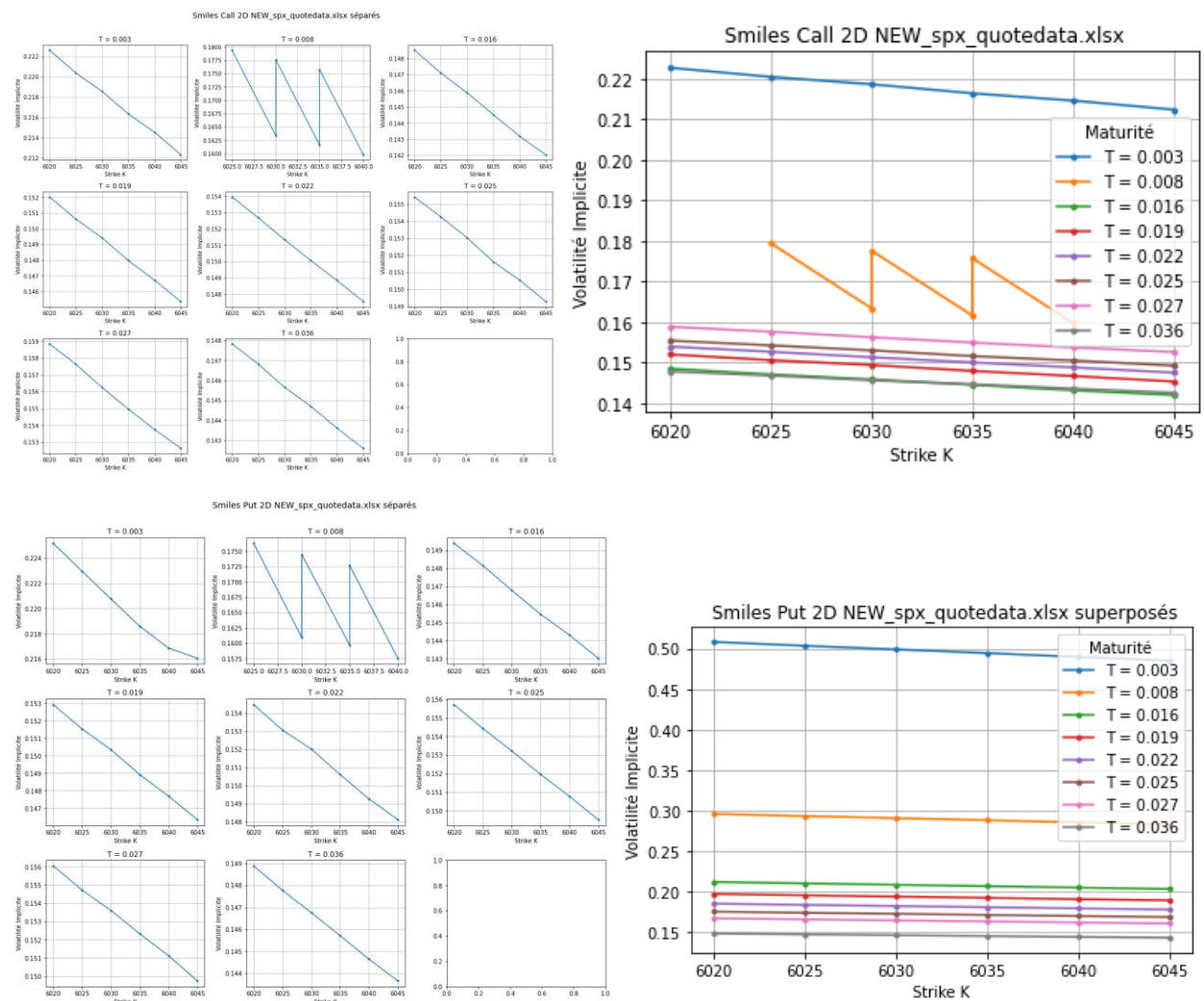
On calcule tout d'abord S_0 et q par régression linéaire, pour ensuite pouvoir tracer nos différents smiles de Call et Put, et enfin nous traçons la droite de régression linéaire.

On obtient donc les valeurs suivantes :

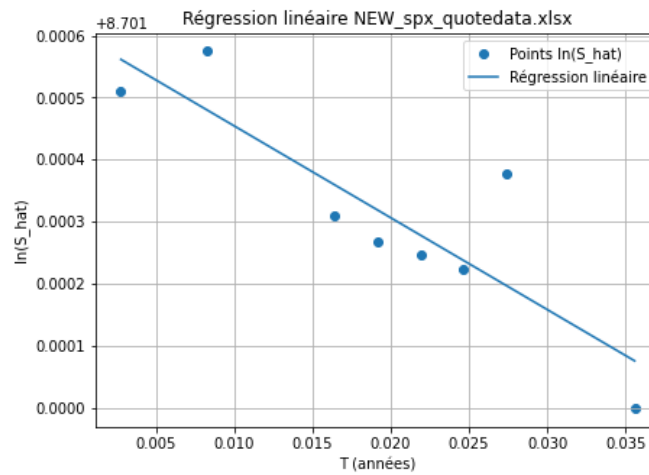
```
Estimation NEW_spx_quotedata.xlsx (régression linéaire via parité
Call-Put)
S0_reg = 6012.536543
q_reg = 0.014791

Estimation par formules théoriques NEW_spx_quotedata.xlsx
beta1 = -0.01479138 donc q = 0.01479138
beta2 = 8.70160199 donc S0 = 6012.53654333
```

Puis les smiles suivants :



Et enfin la droite de régression linéaire suivante :



Question 11) : Analyser les smiles. Conclure.

On observe que quand le strike augmente, la volatilité implicite baisse. Les courbes sont presque droites, on ne voit pas de grandes courbures. On remarque également que les maturités très courtes ont une volatilité plus haute que les maturités un peu plus longues.

Ainsi, les smiles et leur évolution sont cohérents avec ce qu'on observe d'habitude sur le SPX, c'est-à-dire plus de peur à court terme et prime pour la baisse.